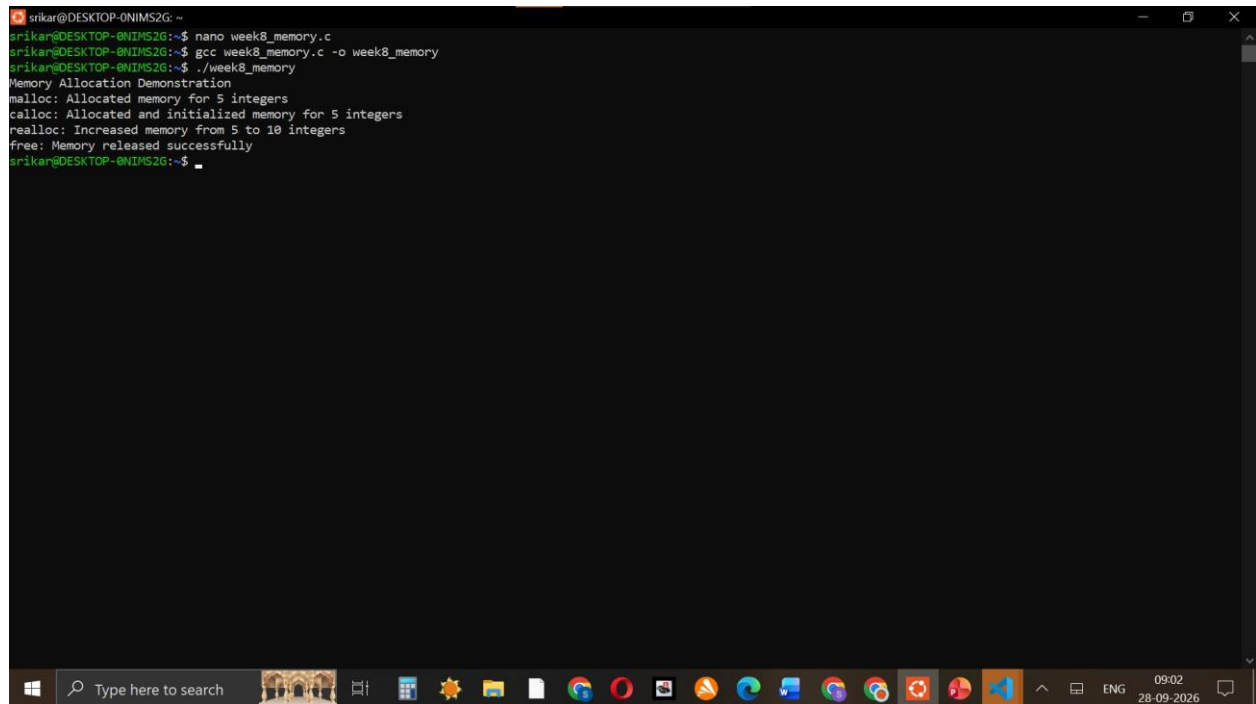


Practical Week-8

1. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.



```
srskar@DESKTOP-0NIMS2G: ~  
srskar@DESKTOP-0NIMS2G:~$ nano week8_memory.c  
srskar@DESKTOP-0NIMS2G:~$ gcc week8_memory.c -o week8_memory  
srskar@DESKTOP-0NIMS2G:~$ ./week8_memory  
Memory Allocation Demonstration  
malloc: Allocated memory for 5 integers  
calloc: Allocated and initialized memory for 5 integers  
realloc: Increased memory from 5 to 10 integers  
free: Memory released successfully  
srskar@DESKTOP-0NIMS2G:~$
```

The screenshot shows a Windows 10 desktop environment. A terminal window is open, displaying the execution of a C program. The program demonstrates memory allocation using malloc, calloc, realloc, and free. The output shows that memory was allocated for 5 integers, then increased to 10 integers using realloc, and finally released successfully. The taskbar at the bottom shows various application icons and the system clock indicating 09:02 on 28-09-2026.

```
srikar@DESKTOP-0NIMS2G: ~  
srikar@DESKTOP-0NIMS2G:~$ nano week8_memory.c  
srikar@DESKTOP-0NIMS2G:~$ gcc week8_memory.c -o week8_memory  
srikar@DESKTOP-0NIMS2G:~$ ./week8_memory  
Memory Allocation Demonstration  
malloc: Allocated memory for 5 integers  
calloc: Allocated and initialized memory for 5 integers  
realloc: Increased memory from 5 to 10 integers  
free: Memory released successfully  
srikar@DESKTOP-0NIMS2G:~$ valgrind --version  
valgrind-3.22.0  
srikar@DESKTOP-0NIMS2G:~$ valgrind --leak-check=full ./week8_memory  
==83144== Memcheck, a memory error detector  
==83144== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.  
==83144== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info  
==83144== Command: ./week8_memory  
==83144==  
Memory Allocation Demonstration  
malloc: Allocated memory for 5 integers  
calloc: Allocated and initialized memory for 5 integers  
realloc: Increased memory from 5 to 10 integers  
free: Memory released successfully  
==83144==  
==83144== HEAP SUMMARY:  
==83144==    in use at exit: 0 bytes in 0 blocks  
==83144==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated  
==83144==  
==83144== All heap blocks were freed -- no leaks are possible  
==83144==  
==83144== For lists of detected and suppressed errors, rerun with: -s  
==83144== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)  
srikar@DESKTOP-0NIMS2G:~$  
  
srikar@DESKTOP-0NIMS2G:~$ nano week8_cow.c  
srikar@DESKTOP-0NIMS2G:~$ gcc week8_cow.c -o week8_cow  
srikar@DESKTOP-0NIMS2G:~$ ./week8_cow  
Parent: Memory allocated and initialized.  
Parent: Address of data = 0x717499cb3010  
Child: Before modification, data[0] = 0  
Child: After modification, data[0] = 999  
Child: Address of data = 0x717499cb3010  
Parent: After child modification, data[0] = 0  
Parent: Address of data = 0x717499cb3010  
srikar@DESKTOP-0NIMS2G:~$
```

Output:

2. Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main() {
    int stack_var = 200;
    int *heap_var = malloc(sizeof(int));
    *heap_var = 300;

    printf("Process PID: %d\n", getpid());
    printf("Global address: %p\n", (void *)&global_var);
    printf("Stack address : %p\n", (void *)&stack_var);
    printf("Heap address  : %p\n", (void *)heap_var);

    printf("\nProcess is running...\n");
    printf("Use another terminal to inspect /proc/%d/maps\n", getpid());

    while (1) {
        sleep(1);
    }

    free(heap_var);
    return 0;
}

imvarls@LAPTOP-VQ4D70L7:  ×  +  ▾

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main() {
    int stack_var = 200;
    int *heap_var = malloc(sizeof(int));
    *heap_var = 300;

    printf("Process PID: %d\n", getpid());
    printf("Global address: %p\n", (void *)&global_var);
    printf("Stack address : %p\n", (void *)&stack_var);
    printf("Heap address  : %p\n", (void *)heap_var);

    printf("\nProcess is running...\n");
    printf("Use another terminal to inspect /proc/%d/maps\n", getpid());

    while (1) {
        sleep(1);
    }

    free(heap_var);
    return 0;
}
```

Output:

```
srikan@DESKTOP-0NIMS2G: ~  
week8_cow.c:23:16: error: expected ')' before 'include'  
23 |         fclose(file #include <sys/wait.h>  
    |                   ^~~~~~  
    |                   )  
week8_cow.c:73:1: error: expected declaration or statement at end of input  
73 | }  
    | ^  
srikan@DESKTOP-0NIMS2G:~$ cat > week8_cow.c  
cat > week8_cow.c  
cat > week8_cow.c  
srikan@DESKTOP-0NIMS2G:~$ gcc week8_cow.c -o week8_cow  
week8_cow.c:1:5: error: expected '=', ',', ';', 'asm' or '__attribute__' before '>' token  
1 | cat > week8_cow.c  
  | ^  
srikan@DESKTOP-0NIMS2G:~$ gcc week8_cow.c -o week8_cow  
week8_cow.c:1:5: error: expected '=', ',', ';', 'asm' or '__attribute__' before '>' token  
1 | cat > week8_cow.c  
  | ^  
srikan@DESKTOP-0NIMS2G:~$ rm week8_cow.c  
srikan@DESKTOP-0NIMS2G:~$ nano week8_cow.c  
srikan@DESKTOP-0NIMS2G:~$ gcc week8_cow.c -o week8_cow  
srikan@DESKTOP-0NIMS2G:~$ ./week8_cow  
Parent: Memory allocated and initialized.  
Parent: Address of data = 0x71b8d339e010  
Parent: Memory before fork: VmRSS: 1936 kB  
Child: Memory before modification: VmRSS: 1200 kB  
Child: Before modification, data[0] = 0  
Child: After modification, data[0] = 999  
Child: Memory after modification: VmRSS: 1392 kB  
Child: Address of data = 0x71b8d339e010  
Parent: After child modification, data[0] = 0  
Parent: Memory after child modification: VmRSS: 2128 kB  
Parent: Address of data = 0x71b8d339e010  
srikan@DESKTOP-0NIMS2G:~$
```